



ORIENTAÇÃO A OBJETOS EM JAVASCRIPT

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

- **NESTA AULA:**

- Capítulo 2 – Visão Geral sobre ECMAScript e JavaScript



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



0



undefined



O QUE É ORIENTAÇÃO A OBJETOS?

- Orientação a Objetos (OO) é um paradigma de programação que organiza o código em torno de objetos, que representam **entidades do mundo real** ou conceitos abstratos.
- **Cada objeto é uma instância de uma classe**, que funciona como um "molde" definindo características (atributos) e comportamentos (métodos).
- Esse paradigma surgiu para tornar o desenvolvimento de **software mais modular, reutilizável e fácil de manter**, aproximando a programação da forma como pensamos o mundo.

CONCEITOS BÁSICOS DE ORIENTAÇÃO A OBJETOS

*Válidos para todas as Linguagens do
paradigma OO*

CLASSES E OBJETOS

- **CLASSE:**

- É o modelo/estrutura que define os atributos (dados) e os métodos (ações) de um objeto.
- Exemplo: Carro pode ser uma classe.

- **OBJETO:**

- É uma instância de uma classe.
- Exemplo: um carro específico como "Carro da Ana, modelo 2020" é um objeto da classe Carro.

ATRIBUTOS (OU PROPRIEDADES) E MÉTODOS

- **ATRIBUTOS** (ou propriedades)
 - São as características do objeto (variáveis).
 - Exemplo: cor, modelo, velocidade.
- **MÉTODOS** (ou funções dentro da classe)
 - São os comportamentos/ações que o objeto pode realizar.
 - Exemplo: `acelerar()`, `frear()`, `buzinar()`.



CONCEITOS SOBRE ORIENTAÇÃO A OBJETOS

Características dessa paradigma

ENCAPSULAMENTO

- ✓ Esconde detalhes internos do objeto, expondo apenas o que é necessário.
- ✓ Protege os dados e controla o acesso.
- ✓ Exemplo: só conseguimos mudar a velocidade de um carro chamando o método **acelerar()** ao invés de alterar diretamente a variável velocidade.



HERANÇA

- ✓ Permite que uma classe (**subclasse**) reaproveite atributos e métodos de outra (**superclasse**).
- ✓ Exemplo: **Carro** é uma classe base, **CarroEsportivo** pode herdar dela e acrescentar novos comportamentos.



POLIMORFISMO

- Objetos diferentes podem responder de maneira distinta à mesma mensagem/método.
- Exemplo: um método `fazerSom()` pode ter resultados diferentes em classes como Cachorro (latir) e Gato (miar).



ABSTRAÇÃO

- **Foca no essencial**, ignorando detalhes desnecessários.
- Exemplo: quando dirigimos, usamos volante e pedais, sem nos preocupar com como o motor funciona internamente.



ORIENTAÇÃO A OBJETOS EM JAVASCRIPT

Aspectos particulares do JavaScript

ORIENTAÇÃO A OBJETOS EM JAVASCRIPT

- O JavaScript é uma linguagem multi-paradigma: dá pra programar de forma imperativa, **funcional e orientada a objetos**.
- No caso de OO, o JS tem uma particularidade: originalmente ele era baseado em protótipos, mas desde o ES6 (2015) ganhou a palavra-chave **class**, que deixou a sintaxe mais próxima de linguagens como Java, C# e Python.



CONCEITOS DE ORIENTAÇÃO A OBJETOS EM JS

- **Classe**

- É um "molde" para criar objetos.
- Define atributos (propriedades) e métodos (funções ligadas ao objeto).

- **Objeto**

- É a instância de uma classe.
- Exemplo: se Carro é a classe, meuCarro é um objeto dela.

- **Construtor**

- Método especial (constructor) usado para inicializar os atributos da classe.

CONCEITOS DE ORIENTAÇÃO A OBJETOS EM JS

- **Encapsulamento**
 - Controle de acesso aos atributos e métodos.
 - Desde o ES2022, temos atributos privados com #.
- **Herança**
 - Feita com `extends`, permitindo que uma classe herde atributos e métodos de outra.
- **Polimorfismo**
 - Uma subclasse pode sobrescrever (*override*) métodos da superclasse.
- **Abstração**
 - Criar classes que representam conceitos gerais, deixando detalhes para subclasses.

EXEMPLO DOS CONCEITOS DE OO EM JAVASCRIPT



```
1 // Classe base
2 class Animal {
3     constructor(nome) {
4         this.nome = nome; // atributo
5     }
6
7     fazerSom() { // método genérico
8         console.log("O animal faz um som...");
9     }
10 }
11
12 // Subclasse
13 class Cachorro extends Animal {
14     fazerSom() { // polimorfismo (sobrescrevendo o método)
15         console.log("Au au!");
16     }
17 }
```

EXEMPLO DOS CONCEITOS DE OO EM JAVASCRIPT

```
19 class Gato extends Animal {  
20     fazerSom() {  
21         console.log("Miau!");  
22     }  
23 }  
24  
25 // Criando objetos  
26 const dog = new Cachorro("Rex");  
27 const cat = new Gato("Mimi");  
28  
29 dog.fazerSom(); // Au au!  
30 cat.fazerSom(); // Miau!
```



ATRIBUTO PRIVADO EM JAVASCRIPT



```
1  class ContaBancaria {
2      #saldo = 0; // atributo privado
3
4      depositar(valor) {
5          this.#saldo += valor;
6      }
7
8      verSaldo() {
9          return this.#saldo;
10     }
11 }
12
13 const conta = new ContaBancaria();
14 conta.depositar(100);
15 console.log(conta.verSaldo()); // 100
16 // console.log(conta.#saldo); Erro! Não pode acessar diretamente
```

MEMBROS ESTÁTICOS

Podem ser usados sem precisar criar objeto

MÉTODOS ESTÁTICOS



```
1  class Matematica {
2      static somar(a, b) {
3          return a + b;
4      }
5
6      static subtrair(a, b) {
7          return a - b;
8      }
9  }
10
11  console.log(Matematica.somar(5, 3));    // 8
12  console.log(Matematica.subtrair(10, 4)); // 6
13
14  const m = new Matematica();
15  // m.somar(1,2); ERRO → métodos estáticos não pertencem ao objeto
```

PROPRIEDADE ESTÁTICA



```
1  class Pessoa {
2      static especie = "Humano"; // atributo estático
3
4      constructor(nome) {
5          this.nome = nome;
6      }
7
8      apresentar() {
9          console.log(`Oi, eu sou ${this.nome} e sou um ${Pessoa.especie}.`);
10     }
11 }
12
13 const p1 = new Pessoa("Ana");
14 p1.apresentar(); // Oi, eu sou Ana e sou um Humano
15 console.log(Pessoa.especie); // Humano
```

MODULARIZAÇÃO EM JAVASCRIPT

Construa a classe uma vez, use sempre

MODULARIZAÇÃO EM JAVASCRIPT

- A modularização consiste em **dividir** um código gigante e bagunçado em pequenos pedaços independentes (módulos). Imagine trocar um "texto de 50 páginas sem parágrafos" por um "livro com capítulos bem definidos". Cada capítulo tem um tema, mas todos juntos formam a história.
- **Reutilização:**
 - Você escreve uma função de validação de CPF uma vez e a usa em dez projetos diferentes.
- **Encapsulamento:**
 - O que acontece dentro do módulo, fica no módulo. Você só expõe o que for necessário.
- **Manutenibilidade:**
 - É muito mais fácil consertar um cano específico na cozinha do que ter que derrubar a casa inteira para achar um vazamento.

TIPOS DE MODULARIZAÇÃO EM JAVASCRIPT

- **CommonJS (CJS)**

- Muito utilizado no Node.js. Ele usa as palavras-chave **module.exports** e **require**.
- Síncrono: Bom para servidores, ruim para navegadores (pois travaria o carregamento).

- **ES Modules (ESM)**

- É o padrão oficial moderno do JavaScript (ES6+). Funciona tanto no navegador quanto no Node.js. Ele usa **import** e **export**.
- Assíncrono: Perfeito para a web.



DECLARA E EXPORTA A CLASSE COM CommonJS



```
1  class Carro {  
2      constructor(modelo, ano) {  
3          this.modelo = modelo;  
4          this.ano = ano;  
5      }  
6  
7      acelerar() {  
8          console.log(`${this.modelo} está acelerando...`);  
9      }  
10 }  
11  
12 // exportando a classe  
13 module.exports = Carro;
```



IMPORTANDO A CLASSE USANDO CommonJS



```
1 // importando a classe
2 const Carro = require('./Carro.js');
3
4 const meuCarro = new Carro("Fusca", 1970);
5 meuCarro.acelerar();
```



DECLARA E EXPORTA A CLASSE COM ES Modules



```
1  // Declarando e exportando a classe
2  export default class Carro {
3      constructor(modelo, ano) {
4          this.modelo = modelo;
5          this.ano = ano;
6      }
7
8      acelerar() {
9          console.log(`${this.modelo} está acelerando...`);
10     }
11 }
```



IMPORTANDO A CLASSE USANDO ES Modules



```
1 // importando a classe
2 import Carro from "./Carro.js";
3
4 const meuCarro = new Carro("Fusca", 1970);
5 meuCarro.acelerar();
```



EXERCÍCIOS

Conceitos de OOP em Prática

EXERCÍCIOS DE FIXAÇÃO

- 1) Crie uma classe **Pessoa** em **Pessoa.js** com os atributos **nome** e **idade** e um método **apresentar()**. No **app.js**, importe a classe, crie dois objetos e chame o método para cada um.
- 2) Crie uma classe **Carro** em **Carro.js** com atributos **modelo**, **ano** e **velocidade**. Implemente os métodos **acelerar()** (aumenta 10 km/h) e **frear()** (diminui 5 km/h). No **app.js**, simule uma corrida de 3 acelerações e 1 freada.
- 3) Crie uma classe base **Animal** em **Animal.js** com um método **falar()**. Depois crie as subclasses **Cachorro** e **Gato** em arquivos separados (**Cachorro.js**, **Gato.js**) que sobrescrevem o método **falar()**. No **app.js**, importe e teste.

EXERCÍCIOS DE FIXAÇÃO

- 4) Crie uma classe **ContaBancaria** em **ContaBancaria.js** com atributo privado **saldo**. Implemente os métodos **depositar(valor)**, **sacar(valor)** e **verSaldo()**. No **app.js**, simule depósitos e saques.
- 5) No arquivo **Veiculos.js**, crie as classes **Carro** e **Moto**. Exporte ambas. No **app.js**, importe as duas e crie objetos testando métodos.
- 6) Crie uma classe **Endereco** em **Endereco.js** e uma classe **Cliente** em **Cliente.js** que recebe um **Endereco** no construtor. No **app.js**, monte um cliente com endereço e exiba os dados.

EXERCÍCIOS DE FIXAÇÃO

- 7) Crie uma classe **Funcionario** em **Funcionario.js** com atributos **nome** e **salario**, e um método **calcularBonus()**. Depois crie classes **Gerente** e **Desenvolvedor** que sobrescrevem esse método para cálculos diferentes. No **app.js**, crie objetos de cada tipo e mostre os bônus.
- 8) Crie uma classe **Matematica** em **Matematica.js** apenas com métodos estáticos **somar(a,b)**, **subtrair(a,b)** e **fatorial(n)**. No **app.js**, importe e use sem precisar instanciar.
- 9) Crie classes **Produto** (com **nome** e **preço**) e **Carrinho** (com lista de produtos e método **total()**). Salve cada uma em arquivos separados. No **app.js**, crie produtos, adicione no carrinho e mostre o valor total.

EXERCÍCIOS DE FIXAÇÃO

10) Crie um Projeto, que contenha:

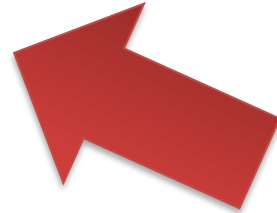
- **Livro.js** → classe com titulo, autor e ano.
- **Biblioteca.js** → classe que mantém um array (vetor) de livros com métodos **adicionarLivro()**, **listarLivros()** e **buscarPorAutor(autor)**.
- **app.js** → monte uma biblioteca com pelo menos 3 livros e faça buscas por autor.



LEITURA PARA A PRÓXIMA AULA

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

– **Capítulo 3 - Arrays**



OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: **Google Class Room**
- Em caso de dúvidas, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br



REFERÊNCIAS DESSA AULA

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora**, 2019.
 - Capítulo 2 – Visão geral sobre ECMAScript e TypeScript